

Mundos Imaginados

Aprendendo a Dinâmica do CarRacing com Tokens Visuais e um GPT Causal

Lucas Barcelos

IMPA Tech / Visgraf

19 de agosto de 2026

Baseado em “World Models” (Ha & Schmidhuber, 2018)

- 1 Visão Geral
- 2 Coleta de Dados
- 3 Encoder — VQ-VAE
- 4 Primeira Versão — GPT Image
- 5 Modelo de Dinâmica — GPT
- 6 Resultados
- 7 Escolhas de Design
- 8 Passos Futuros
- 9 Demo

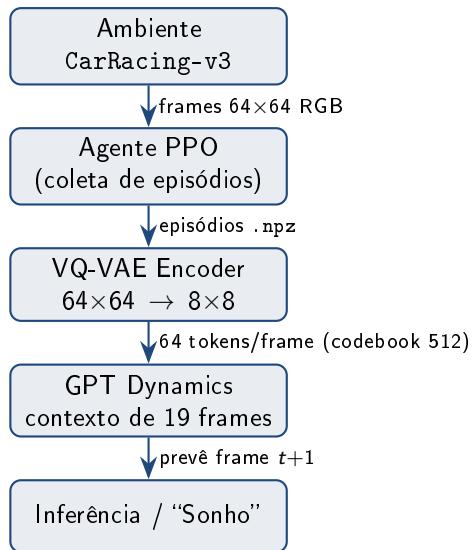
Objetivo: aprender um *world model* do ambiente CarRacing-v3 — um modelo que simula (“sonha”) o próximo frame do jogo condicionado à ação do agente.

Abordagem em dois estágios

- 1 **Encoder perceptivo discreto** (VQ-VAE): comprime cada frame em um conjunto de *tokens visuais*.
- 2 **Modelo de dinâmica autorregressivo** (GPT causal): prevê a sequência de tokens do próximo frame dado o contexto e a ação.

Discretizar a imagem transforma a previsão de frames em um problema de **modelagem de linguagem** — trocamos regressão contínua por *cross-entropy* sobre tokens.

Pipeline Completo



agente_coleta/coleta.py

Um agente **PPO** pré-treinado (Stable-Baselines3) joga o CarRacing-v3 com **5 ações discretas** (esquerda, direita, acelerar, frear, neutro).

Para cobrir estados raros, aplica-se um cronograma de exploração **ϵ -greedy crescente**: a maior parte do dataset é de alta qualidade, mas uma fração introduz diversidade.

ϵ	Fração
0.02	25 %
0.05	25 %
0.10	20 %
0.20	15 %
0.35	10 %
0.60	5 %

Cronograma de exploração

Cada episódio passa por um *wrapper stack* antes de ser salvo:

- `FrameSkip(4)` — acumula recompensa em 4 frames; reduz correlação temporal.
- `CropBlackBar(12)` — remove a barra de informações inferior.
- `ResizeObservation(64×64)` — redimensiona o frame.
- `NormalizeAndTranspose` — $(H, W, C) \text{ uint8} \rightarrow (C, H, W) \text{ float32}$ em $[0, 1]$.
- `FrameStackChannels(4)` — empilha 4 frames no eixo de canais (entrada do PPO).

Formato de saída

Cada episódio $\rightarrow$ `.npz` com `obs( $T, 3, 64, 64$ )`, `actions( $T, \cdot$ )`, `rewards( $T, \cdot$ )` e `dones( $T, \cdot$ )`.

`models/encoder/modules.py`, `models/encoder/train.py`

Comprime cada frame $64 \times 64 \times 3$ em uma grade $8 \times 8 = 64$ **tokens discretos**, índices num **codebook de 512** entradas.

Encoder ($64 \times 64 \rightarrow 8 \times 8$)

- Conv $3 \times 3 \rightarrow 64$ canais
- 4 estágios, `ch_mult=[1, 2, 4, 4]`, 2 ResnetBlocks cada
- Atenção espacial em 16×16 e 8×8
- 3 Downsamples $\rightarrow$ `latent_dim=256`

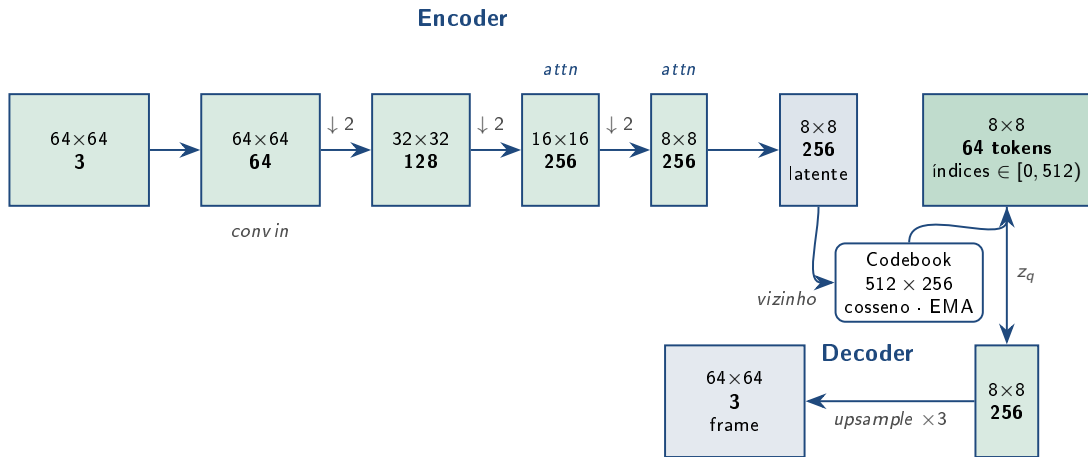
Quantização Vetorial

- Codebook 512×256
- Similaridade de cosseno
- EMA decay = 0.99
- *commitment weight* = 0.25

Decoder ($8 \times 8 \rightarrow 64 \times 64$)

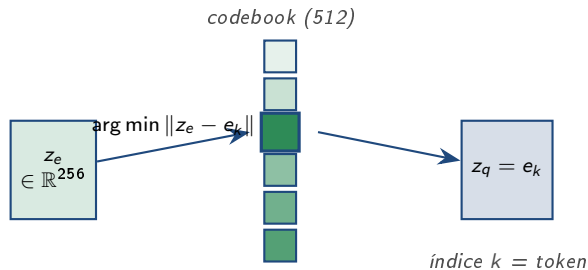
- Simétrico, `ch_mult=[4, 4, 2, 1]`
- Upsample (nearest + conv)

VQ-VAE — Arquitetura (diagrama)



Cada frame $\rightarrow$ grade 8×8 de **64 tokens**; resolução cai $4 \times$ e a profundidade sobe. Atenção só em 16×16 e 8×8 .

Quantização Vetorial — Mecanismo



Passos

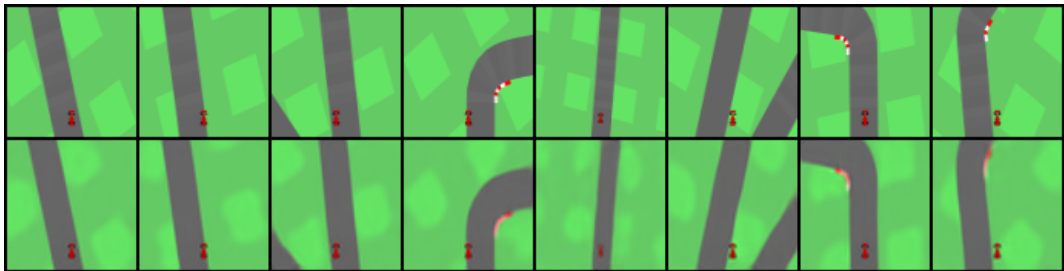
- 1 Encoder produz vetor contínuo z_e por posição.
- 2 Escolhe a entrada mais próxima e_k do codebook (cosseno).
- 3 O **índice** k é o token; $z_q = e_k$ vai ao decoder.

Treino

$$\mathcal{L} = \underbrace{\|x - \hat{x}\|^2}_{\text{recon (MSE)}} + 0.25 \underbrace{\|z_e - \text{sg}[e_k]\|^2}_{\text{commitment}}$$

Codebook atualizado por **EMA** (0.99);
gradiente passa por *straight-through*.

VQ-VAE — Reconstrução



Topo: frame original | Base: reconstrução do VQ-VAE (época 84, melhor `val_loss`). A geometria da pista e a posição do carro são preservadas; detalhes finos ainda perdem nitidez.

Treino: `MSE + VQ commitment · AdamW (lr  $10^{-4}$ ) · batch 128 · orçamento de 400 épocas, parou em 84 · AMP · grad clip 1.0`

Por que medir?

Um codebook saudável usa **muitos dos 512 códigos** e evita *codebook collapse* (poucos códigos concentrando quase todo o uso). Isso se mede por:

- códigos ativos / 512
- **perplexity** = $e^{H(p)}$ — quanto mais perto de 512, mais uniforme o uso
- % dos tokens coberto pelos 10 códigos mais frequentes

Como medir

`results/plot_codebook_usage.py` calcula essas três métricas direto de `dataset_tokens/` (ou codificando frames crus com `-encode`).

GPT Image — Primeira Versão do Modelo de Dinâmica

`models/dynamics/gptimage.py, trainimage.py`

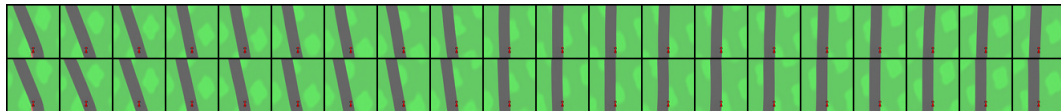
Antes de fechar o world model como MDP completo, a primeira versão só validava a parte mais arriscada: **um Transformer causal consegue prever os tokens visuais do próximo frame?**

O que essa versão fazia

- Mesmo layout de sequência (19 frames de contexto, blocos de 65 tokens = 64 visuais + 1 ação).
- **Só previa os 64 tokens do próximo frame** — sem cabeça de recompensa ou de done.
- Objetivo: isolar e validar a dinâmica visual antes de somar a complexidade de recompensa/done/RL.

Depois de validado, essa base virou `gptdynamics.py`: mesmo esqueleto, mais as cabeças de recompensa e done (próximos slides).

GPT Image — Resultados da Primeira Versão



Topo: sonho (só tokens visuais) | Base: real.

Métrica (rollout 20f)	Valor
PSNR inicial (f. 0)	40,0 dB
PSNR final (f. 19)	24,9 dB
PSNR médio	30,9 dB

Fidelidade visual alta confirmou que a abordagem (VQ-VAE + Transformer causal) funcionava — justificou investir na versão completa com recompensa/done.

GPT Dynamics — Layout de Sequência

`models/dynamics/gptdynamics.py`, `traingpt.py`

Evolução direta da primeira versão (`gptimage.py`): mesmo esqueleto de sequência, agora com recompensa e done.

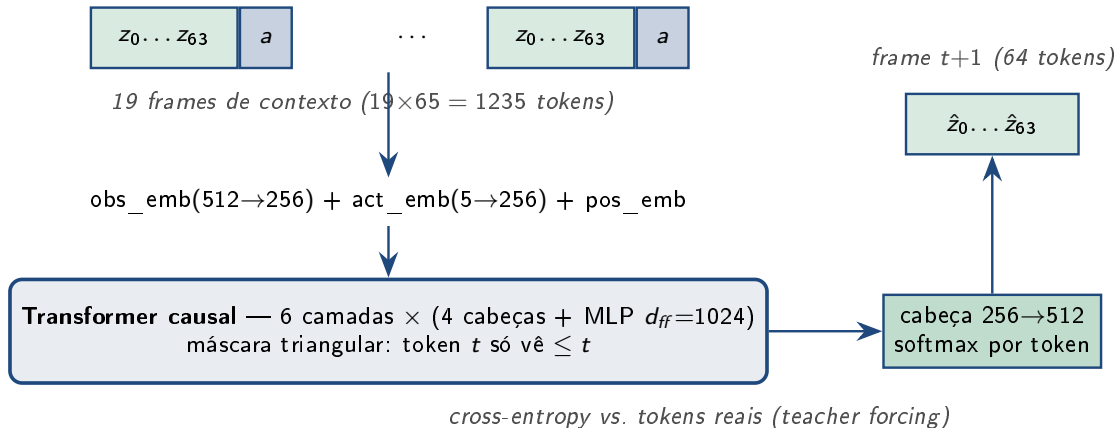
O modelo prevê os **64 tokens** do próximo frame e a recompensa/done da transição, dado um contexto de **19 frames** + a ação a executar. Cada frame é um *bloco de 65 tokens*:

$$\underbrace{z_0, z_1, \dots, z_{63}}_{64 \text{ tokens visuais}}, \underbrace{a}_{1 \text{ ação}}$$

Dimensões

- Sequência de entrada: $19 \times 65 = 1235$ tokens.
- `block_size` máximo: $20 \times 65 = 1300$ (contexto + bloco alvo).
- A representação da **última posição do contexto** alimenta, além da cabeça de tokens, uma cabeça de **recompensa** (3 classes: neg/neutro/pos) e uma de **done** (2 classes).

GPT Dynamics — Como o Transformer prevê



O modelo é um **GPT**: prediz o próximo token do stream, condicionado no contexto e na ação a . A mesma representação também alimenta `head_rewards` ($256 \rightarrow 3$) e `head_dones` ($256 \rightarrow 2$).

Arquitetura

Componente	Detalhe
obs_emb	512 $\rightarrow$ 256 (amarrado à cabeça)
act_emb	5 $\rightarrow$ 256
pos_emb	aprendido, 1300 pos.
Transformer	6 cam., 4 cab., $d_{ff}=1024$ GELU, norm_first
Máscara	causal (triangular)
Cabeça obs	256 $\rightarrow$ 512 (tied)
Cabeça reward	256 $\rightarrow$ 3 (neg/neutro/pos)
Cabeça done	256 $\rightarrow$ 2

~4 M de parâmetros

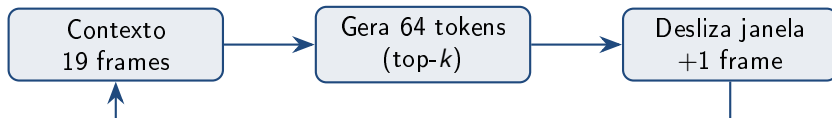
Treinamento

Hiperparâm.	Valor
Loss	CE token + CE reward + CE done
Otimizador	AdamW, wd 0.01
lr	10^{-4}
Batch	32
Épocas	orçamento 5000, parou em 18
Precisão	AMP
Grad clip	1.0
Contexto	19 frames

Teacher forcing: concatena o frame alvo ao contexto e prevê cada token com shift de posição no bloco.

`imagine_next_frame`

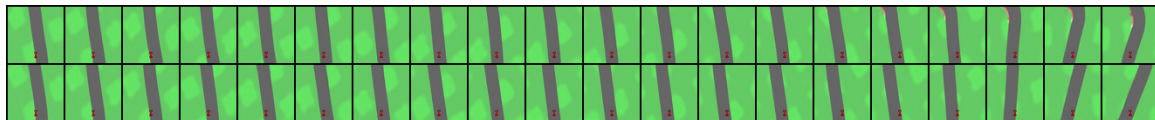
- Os 64 tokens do próximo frame são gerados **um a um**; a classe de **recompensa** e o **done** da transição saem no mesmo passo (antes do primeiro token visual, via KV-cache).
- **Top- k sampling** ($k = 50$, temperatura = 1.0) — evita tokens de baixíssima probabilidade sem colapsar para *greedy*.
- **Janela deslizante**: a cada frame gerado, descarta-se o mais antigo e insere-se o recém-criado, mantendo sempre 19 frames de contexto.



Resultados — Sonho vs. Real

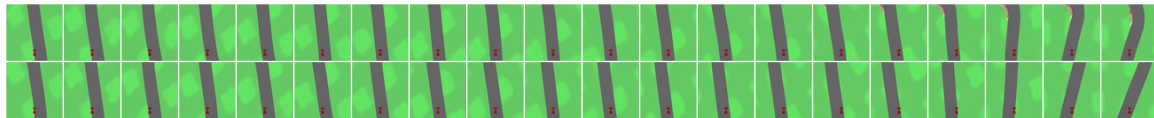
`models/dynamics/infergpt.py`

Seed de **19 frames** reais → modelo “sonha” **40 frames** usando as ações reais do episódio.



Linha superior: **sonho** do modelo | Linha inferior: **trajetória real**.

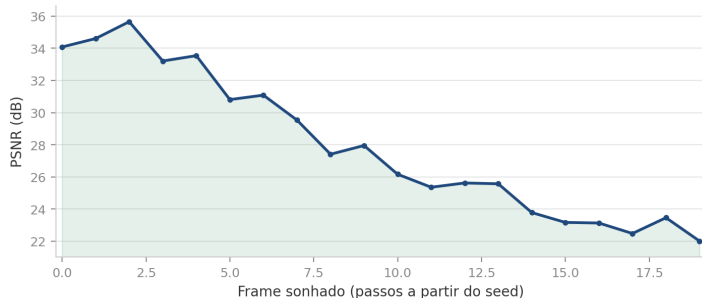
Resultados — Rollout Sonhado (frames dos GIFs)



Topo: **sonho** do modelo (rollout autorregressivo) | Base: **real**. Frames amostrados de `dream.gif` / `real.gif` (`infergpt.py`).

GIFs animados disponíveis em `results/dreams/` e frames individuais em `results/slides_assets/`.

Resultados — Degradação Quantitativa



PSNR calculado frame-a-frame entre `dream.gif` e `real.gif` (`infergpt.py`).

Métrica (rollout 20f)	Valor
PSNR inicial (f. 0)	34,1 dB
PSNR final (f. 19)	22,0 dB
PSNR médio	27,9 dB
Queda por frame	−0,64 dB

O sonho parte quase idêntico ao real e **diverge de forma suave e monótona** — coerente com a inspeção visual da filmstrip.

Observações qualitativas

O modelo mantém a **estrutura global da pista** — geometria da curva, posição do carro, coloração do gramado — ao longo dos frames sonhados, com *degradação gradual* dos detalhes finos à medida que a predição se afasta do seed.

Sanidade quantitativa (rollout de 20 frames, 1 episódio de teste)

- **Token accuracy**: 33,1% dos 64 tokens sonhados coincidem exatamente com os reais (chance $\approx 0,2\%$ em 512 códigos).
- **Acurácia de recompensa** (3 classes): 72,5% (chance = 33%).

Saídas geradas: `dream.gif` (sonho), `real.gif` (real) e `comparison.png` (grid comparativo).

Decisão	Motivação
Discretização VQ-VAE	Tokens simplificam a previsão do GPT (cross-entropy vs. regressão); segue IRIS (Micheli et al., 2023) — VQ-VAE + Transformer, diferente do VAE contínuo + RNN de Ha & Schmidhuber (2018).
Atenção só em baixa resolução	Reduz custo quadrático mantendo capacidade global (8×8 , 16×16).
Pesos amarrados (tied)	Regularização padrão; reduz parâmetros sem perder expressividade.
ε -schedule crescente	Cobre estados raros sem sacrificar a qualidade média do dataset.
Teacher forcing	Estabilidade de gradiente; evita BPTT longo.
Top- k sampling	Evita tokens improváveis sem colapsar para greedy.
Reward/done como classificação	Cross-entropy sobre classes discretas (estilo IRIS), não regressão symlog (estilo DreamerV3).

Passos Futuros

Estender o world model offline

- Treinar por mais épocas — fidelidade do sonho e uso do codebook ainda têm bastante margem pra melhorar.
- Estender o horizonte de sonho e medir *drift* acumulado.

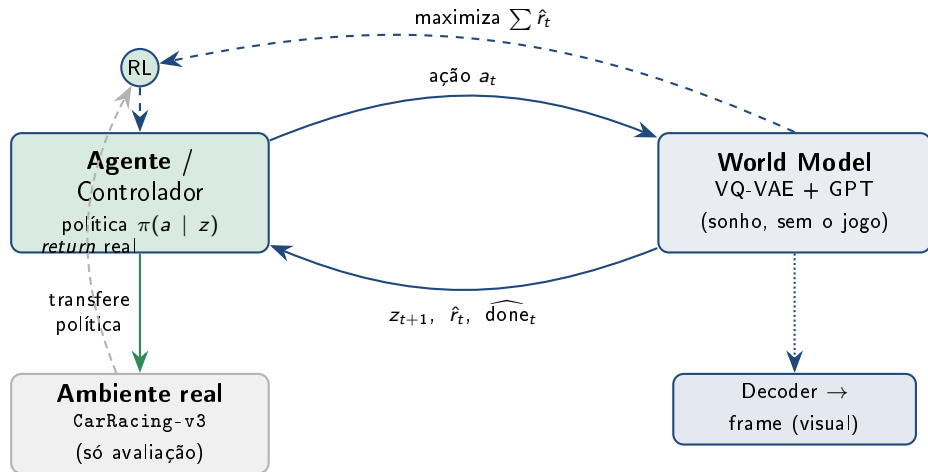
Aprendizado no “sonho” (em andamento)

- Já em desenvolvimento: um **loop online** que co-treina world model e um controlador PPO **inteiramente dentro do sonho** (à la Ha & Schmidhuber / IRIS), transferindo a política pro ambiente real.
- Próximo passo: estabilizar esse loop em horizontes longos de treino.

Avaliação e escala

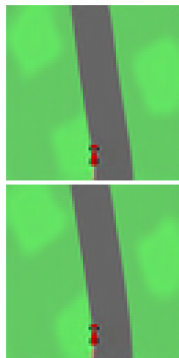
- Métricas de fidelidade perceptual (LPIPS, FID) além de token accuracy.
- Escalar contexto / parâmetros do GPT e refinar o codebook do VQ-VAE.

Passos Futuros — O Ciclo do World Model



O agente é treinado **inteiramente dentro do sonho** — sem rodar o jogo — e só usa o CarRacing real pra *avaliar* a política transferida (à la Ha & Schmidhuber, 2018). Já em desenvolvimento (`train_online.py`).

Demo — Rollout Animado (Sonho vs. Real)



A animação roda no **Adobe Acrobat/Reader** (ou pdfpc); em outros leitores aparece o primeiro frame.

Obrigado!

Mundos Imaginados — VQ-VAE + GPT Dynamics

Lucas Barcelos